ML Integration Guide

Road Disruption Intelligence - how to add machine learning to this repository

Repository: **github.com/AnandRawat138/road-disruption-intelligence** (branch **main**)
Prepared for: the ML team working with Aviral Consulting (OPC) Private Limited
Codebase version: v1.1.0, 25 October 2026 (guide revision 2)

How to use this document. Upload this PDF to your Claude or ChatGPT project together with access to the repository (or the relevant files). Section 16 contains a ready-to-paste instruction for the AI assistant. Sections 2-4 explain the code and the data; **section 5 describes the ML that already exists**; sections 6-13 say exactly **what** to build next, **where** to put it and **how** to wire it in; sections 14-15 define the evaluation, tests and acceptance criteria.

What changed since revision 1 (v1.0.0): version 1.1.0 ships a **baseline ML classifier** (backend/rdi/services/ml/model.py: TF-IDF + logistic regression for relevance, category and time status), an operator setting to classify with the **LLM or the ML model**, and an **automatic ML fallback** when the LLM is unavailable. scikit-learn is now a normal dependency. Do not rebuild these from scratch - extend them (section 5). Still open: a gold evaluation set, advisory relevance scoring of all articles, the category cross-check and embeddings.

Section	Contents
1	Goals and ground rules
2	Repository map - where everything lives
3	How a record flows through the system today
4	The data and the label situation (read before training anything)
5	What already exists in v1.1.0: the built-in ML classifier, engine switch and LLM fallback
6	Remaining ML work, in priority order
7	Target architecture: extending the rdi.services.ml package
8	Database changes and migration
9	Task 1 - Relevance scoring for triage (advisory)
10	Task 2 - Category cross-check
11	Task 3 - Embeddings for duplicates and incident linking
12	API, settings, background jobs and user interface
13	Dependencies, Windows setup and model artifacts
14	Evaluation protocol and model card
15	Tests and definition of done
16	Ready-to-paste prompt for Claude / ChatGPT
17	Pitfalls to avoid

1. Goals and ground rules

The application classifies news articles with a large language model (LLM) that the client controls (local Ollama or a cloud key) or, since v1.1.0, with a small built-in ML model. The LLM is slow on local hardware (roughly 8-15 seconds per article on a 6 GB GPU) and about 16,900 of 17,400 articles are still unclassified. Machine learning should make the system **faster, cheaper and more consistent** - not replace human review.

Primary goals for the ML team

- A **gold evaluation set** so that every model - LLM, ML and the keyword pre-filter - can be measured honestly.
- Fast **relevance scores** for every article (the v1.1.0 relevance model already exists) so the LLM and the operators see what matters first.
- A **category model** that cross-checks the LLM and flags disagreements for review.
- **Text embeddings** that improve duplicate detection and the linking of related reports into one incident.
- A repeatable **training, evaluation and deployment** path that the client can run on a Windows PC without a GPU.

Ground rules (non-negotiable - the existing code follows them and reviewers will check them)

- Do not fabricate labels, metrics or results. Every reported number must come from code that was actually run.
- ML output never auto-approves a record and never deletes data. It may prioritise, pre-fill and flag.
- ML never replaces an LLM classification or a human correction. The v1.1.0 engine/fallback stores ML results as classification versions with provider **ml** only for records that have no LLM/human result (or when the operator explicitly reclassifies with ML). Advisory scores of new tasks go to their own table with the model version.
- The application must keep working when no model is trained or a model file cannot be loaded (graceful degradation).
- No new network services, no GPU requirement, no heavy frameworks (no PyTorch/TensorFlow) unless clearly justified.
- All 75 existing tests must keep passing; new behaviour needs new tests that run without network access.
- Keep the original files in `data/` untouched; never commit secrets, the runtime database or model files larger than ~20 MB.

2. Repository map - where everything lives

```
road-disruption-intelligence/                (repository root)
  data/                                      historical news data, JSONL, read-only input
  README.md                                  install + quick user guide
  road-disruption-intel/
    setup_and_start.bat  start.bat  stop.bat  setup_llm.bat  diagnose.bat
    .env.example                             configuration template (.env is never committed)
    backend/
      requirements.txt                       runtime deps (FastAPI, SQLAlchemy, httpx, feedparser...)
      requirements-dev.txt                   + pytest
      rdi/                                   <-- THE APPLICATION (v1)
        config.py                            settings from env (.env), runtime paths
        main.py                              FastAPI app factory, routers, security middleware
        server.py / cli.py                   entry points (python -m rdi.server / rdi.cli ...)
        db/models.py                         SQLAlchemy ORM models (articles, classifications, incidents...)
        db/session.py                        engine, init_db(), SCHEMA_VERSION, MIGRATIONS, backups
        api/records.py                       /api/records (filters, sort, export, review, corrections)
        api/incidents.py                     /api/incidents, /api/map/incidents
        api/dashboard.py                     /api/dashboard/summary, /api/quality, /api/reports/summary
        api/config_api.py                    /api/llm/*, /api/ml/*, /api/settings/*, /api/system/*, classification tr
        api/highways.py                      /api/highways (per-highway view of open events)
        api/operations.py                    sources, imports, jobs, alerts, audit
        services/ingest.py                   Ingestor: dedup + insert of every new article  (ML HOOK #1)
        services/prefilter.py                keyword pre-filter (the thing the relevance model improves)
        services/classification/schema.py    LLM prompt, output schema, validation
        services/classification/classifier.py classify_article() -> LLM or classify_with_ml(), fallback,
                                             review_reasons()                          (ML HOOK #2)
        services/ml/model.py                 EXISTING ML: training, metrics, load, predict_output()
        services/ml/seed_labels.jsonl        bundled labelled headlines + weak negatives
```

```
    services/highways.py                highway-name extraction (reused by the ML field extraction)
    services/incidents.py               incident linking: find_match()            (ML HOOK #3)
    services/jobs.py, job_handlers.py   background job runner and job types       (ML HOOK #4)
    services/settings_store.py          persisted settings with DEFAULTS
    services/llm/providers.py           Ollama / OpenAI-compatible / Anthropic clients
    services/geo/                       gazetteer + Nominatim geocoder
    services/connectors/                news sources
  tests/                                pytest suite (conftest.py creates a throw-away DB)
    test_5_highways_ml.py               existing ML / fallback / highways tests
  app/, seed_*.py, roaddb.db            legacy v0 code - DO NOT USE, reference only
frontend/
  src/app/App.tsx                       navigation (NAV array) and routes
  src/app/pages/*.tsx                   one file per page (Records.tsx, Highways.tsx, ...)
  src/app/pages/LLMSettings.tsx         "Classification engine" page incl. EnginePanel (ML status/metrics)
  src/app/components/RecordDrawer.tsx   record detail panel
  src/app/api.ts, meta.ts               API client, labels
  dist/                                 pre-built UI (rebuild with: npm ci && npm run build)
runtime/                                created at runtime, gitignored: rdi.db, logs, backups, secrets,
                                        models/ml_classifier.joblib + .json (trained ML model)
docs/                                   USER_GUIDE.md, API.md, screenshots
```

New ML code goes into **backend/rdi/services/ml/** next to the existing model.py (runtime inference), **backend/ml_training/** (offline evaluation scripts) and **backend/tests/test_6_ml_*.py**. Trained models go to **runtime/models/** (not committed). Small edits are needed in the hook files listed above.

3. How a record flows through the system today

Step	Code	What happens
1 Ingest	services/ingest.py: Ingestor.ingest()	Article arrives from a file import or a news connector. Deduplicated by canonical URL, external id, content hash and same-source title. Keyword pre-filter sets prefilter_label = LIKELY_RELEVANT / POSSIBLY_RELEVANT / UNLIKELY.
2 Queue	services/job_handlers.py: _classify()	Operator (or auto-classify) starts a 'classify' job. Records are selected by filters, usually prefilter = LIKELY_RELEVANT, newest first.
3 Classify	services/classification/classifier.py: classify_article()	Setting classification.engine = llm or ml. LLM: returns JSON (relevance, category, location, time, severity, evidence). ML: classify_with_ml() builds the same JSON shape from the ML model (section 5). Both are validated by the same validate_output() and stored as a Classification version with provider/model/prompt_version. If the LLM is unavailable and ml_fallback is on, records without an LLM/human result are classified by ML and labelled.
4 Geo	services/geo/geocoder.py	City/locality resolved to an approximate centroid (gazetteer or Nominatim).
5 Review flags	classifier.review_reasons()	Low confidence, unverified evidence, unresolved location etc. set needs_review.
6 Incident	services/incidents.py: link_article() / find_match()	RELEVANT records are linked to an existing incident (same category + city + time window + matching road/locality) or create a new one.
7 Human review	api/records.py: /review, PATCH /classification	Operator approves/rejects (review_status) or corrects fields (new HUMAN classification version). Approval confirms the incident.

ML already plugs into step 3 (engine choice and fallback). Further work plugs into steps 1, 2, 5 and 6. The LLM remains the better extractor of structured details (roads, times, severity), and ML never performs step 7.

4. The data and the label situation (read before training anything)

4.1 What is in the database

Table / field	Meaning for ML
articles.title, summary_text, body_text	Model input text. Historical and Google News records are mostly **headline only** (summary = headline + publisher).

Table / field	Meaning for ML
articles.source_name, published_at, connector, language	Useful metadata features / split keys.
articles.prefilter_label/score	Keyword rule output. A baseline to beat, NOT a label.
articles.dataset_flags	Provenance from old files, e.g. prior_lexical_score, prior_eval_split. NOT labels.
classifications (is_current=1, origin='MODEL')	LLM output: relevance, category, severity, city... = **silver labels** (noisy, produced by llama3:8b).
classifications (origin='HUMAN')	Operator corrections = **gold labels** for the corrected fields.
articles.review_status = APPROVED / REJECTED	Operator decision on the current classification = **gold** relevance signal (APPROVED: the classification was right; REJECTED: wrong/irrelevant).
incidents + articles.incident_id	Groups of reports about the same event - useful for embedding / linking evaluation and for group-aware splits.

4.2 The honest label situation (25 Oct 2026, operational database)

Signal	Count	Use
Unique articles	17,398 (16,949 unclassified)	Unlabelled pool
LLM classifications (current, llama3:8b, prompt v1.1)	449: 427 RELEVANT, 10 RELATED, 12 IRRELEVANT	Silver training labels (very few negatives)
Operator APPROVED / REJECTED	1 / 0	Gold - must be created
HUMAN corrections	0	Gold - must be created
services/ml/seed_labels.jsonl	449 silver labels + 850 weak negatives	Bootstrap for fresh installs (see 5.2)
data/weak_labels file	8,914 rows	Contains NO labels (only constant 0.85 confidence) - do not use as labels
data/evaluation splits	6,239 / 1,337 / 1,338	No ground truth inside - only a partition of keyword-relevant articles

Consequence: there is currently no ground truth to measure accuracy. Before claiming any metric the team must create a **gold evaluation set**: at least 300-500 articles reviewed by operators in the application (Classification queue: Approve / Reject / Edit fields), sampled across time, sources, cities and pre-filter labels (include UNLIKELY articles, otherwise recall cannot be measured). Train on silver + gold-train, evaluate only on gold-test.

4.3 Exporting a training dataset

Add `backend/ml_training/export_dataset.py` that reads the SQLite database through the existing session (so paths come from `rdi.config.settings`) and writes JSONL to `runtime/ml/datasets/`:

```
# backend/ml_training/export_dataset.py  (run from backend/:  ..\.venv\Scripts\python -m ml_training.export_dataset)
import json
from sqlalchemy import select
from rdi.config import settings
from rdi.db.session import SessionLocal
from rdi.db.models import Article, Classification
from rdi.services.classification.classifier import article_source_text

def label_source(a: Article, c: Classification | None) -> str:
    if c is not None and c.origin == "HUMAN":
        return "gold_corrected"
    if a.review_status in ("APPROVED", "REJECTED"):
        return "gold_reviewed"
    return "silver_llm" if c is not None else "unlabelled"

def main():
    out = settings.runtime_dir / "ml" / "datasets" / "articles.jsonl"
    out.parent.mkdir(parents=True, exist_ok=True)
    with SessionLocal() as db, open(out, "w", encoding="utf-8") as fh:
        rows = db.execute(select(Article, Classification).outerjoin(
            Classification, Classification.id == Article.current_classification_id)
            .where(Article.near_duplicate_of.is_(None)))
        for a, c in rows:
            relevance = c.relevance if c else None
            if a.review_status == "REJECTED":
                relevance = "IRRELEVANT"          # operator says the classification is wrong
            fh.write(json.dumps({
                "id": a.id, "text": article_source_text(a), "source": a.source_name,
                "published_at": a.published_at.isoformat() if a.published_at else None,
                "incident_id": a.incident_id, "prefilter": a.prefilter_label,
                "relevance": relevance, "category": c.category if c else None,
                "label_source": label_source(a, c),
                "classifier": f"{c.provider}/{c.model}/{c.prompt_version}" if c else None,
            }, ensure_ascii=False) + "\n")
    print("wrote", out)

if __name__ == "__main__":
    main()
```

Note on REJECTED: a rejection means the operator disagrees with the current classification. For relevance it is safest to treat it as 'not RELEVANT'; for category training, drop rejected rows rather than guessing the right category.

5. What already exists in v1.1.0: the built-in ML classifier, engine switch and LLM fallback

Version 1.1.0 contains a deliberately simple, working ML layer. Read `backend/rdi/services/ml/model.py` (about 300 lines) before writing anything new - most of the plumbing below is reusable.

5.1 Model

Item	Implementation
Features	FeatureUnion of TF-IDF word 1-2-grams and character 3-5-grams (sublinear tf)
Classifier	LogisticRegression(C=5, class_weight='balanced'); three separate pipelines
Tasks	relevance (RELEVANT vs NOT_RELEVANT); category of relevant records (classes with < 5 examples are not learned); time status (UPCOMING / ONGOING / RESOLVED / UNKNOWN)
Input text	model_text(): headline without publisher suffix, plus the summary only when it is not just the headline again
Output	predict_output() returns the same JSON shape as the LLM, so the existing validate_output(), geocoding, review flags and incident linking are reused unchanged
Fields it fills	relevance, category, time_status, confidence (from probabilities), summary/evidence = headline (verbatim), highway_number and road_name via services/highways.py, city/state via the gazetteer - after masking highway names so 'Mumbai-Goa highway near Ratnagiri' gives Ratnagiri
Fields it never fills	severity (UNKNOWN), event_start/event_end (null) - listed as missing

Item	Implementation
Artifact	runtime/models/ml_classifier.joblib (all three pipelines) + ml_classifier.json (metadata and metrics); cached in memory by file mtime
Relevance decision	P(relevant) >= classification.ml_relevance_threshold (default 0.6) -> RELEVANT; within 0.2 below -> NEEDS_REVIEW; else IRRELEVANT

5.2 Training data and metrics

- **Labels:** current VALID classifications that are not from the ML model itself (LLM and HUMAN). Approved records and human corrections weigh 3x. Rejected records are excluded (a rejection can mean wrong category, irrelevant or duplicate).
- **Seed set:** `seed_labels.jsonl` - 449 llama3:8b/operator labels and 850 weak negatives, so a fresh install can train before it has labels. Database labels take precedence (deduplicated by normalised title).
- **Weak negatives:** unclassified articles the keyword pre-filter rates UNLIKELY, added until negatives = 2 x positives. They make the relevance metric look better than it is - see the 'labelled rows only' column below.
- **Training** runs automatically the first time ML is needed, or via the 'Retrain now' button / POST /api/ml/train (job type 'ml_train'). About 6-10 seconds on the client PC.

Metrics of the model trained on 25 Oct 2026 (`tfidf-logreg-20261025071328`, 1,299 rows), stratified 5-fold cross-validation, i.e. **agreement with the training labels, not accuracy against ground truth**:

Task	n	Accuracy	Macro-F1	Majority baseline	Note
Relevance (all rows)	1,299	96.2%	0.956	67.1%	inflated by easy weak negatives
Relevance (LLM/human-labelled rows only)	449	91.1%	0.631	-	only 22 negatives - weakest point
Category	424	75.2%	0.582	46.0%	OTHER not learned (< 5 examples)
Time status	427	82.9%	0.629	63.7%	

5.3 Engine switch and fallback

Setting (settings_store 'classification')	Effect
engine = 'llm' | 'ml'	Which classifier classify_article() and classify jobs use
ml_fallback = true (default)	If the LLM call fails (unreachable, timeout, auth, quota, config), the failed attempt is stored and the record is classified by ML - but only if it has no valid LLM/human classification yet
ml_relevance_threshold = 0.6	See 5.1

- Jobs probe the LLM once at start (_llm_reachable()); if it is down the whole job runs on ML and says so. If the LLM fails mid-job the job switches to ML for the remaining records. Each ML record carries a validation note explaining why.
- Provenance: provider = 'ml', model = 'tfidf-logreg-<timestamp>', prompt_version = 'ml-v1'; raw_output holds the probabilities. Records produced this way are easy to find and reclassify with the LLM later.
- API: GET /api/ml/status, POST /api/ml/train, POST /api/ml/test; 'engine' parameter on POST /api/classification/run.
- UI: sidebar 'Classification engine' (EnginePanel in LLMSettings.tsx: switch, fallback, threshold, metrics, retrain, test box); 'ML model' option in the record panel's Reclassify list.
- Tests: backend/tests/test_5_highways_ml.py (training, grounded predictions, engine, fallback, job switch, no replacement of LLM results, API).

Known limitations to fix next: (1) no gold labels, so no real accuracy figure; (2) relevance has only 22 labelled negatives - the weak negatives come from the keyword rule, so the model partly re-learns that rule; (3) probabilities are not calibrated; (4) a single model file, no versions/activation history; (5) the ML relevance score is computed only

when a record is classified, not for the 16,900 unclassified articles.

6. Remaining ML work, in priority order

#	Task	Why it matters	Model / starting point	Needs
0	Gold evaluation set	Without it no model (LLM, ML or keyword rule) can be measured	Operator review in the app + export (section 4.3)	>= 300-500 reviewed articles incl. UNLIKELY ones
1	Relevance triage scores	Score all ~16,900 unclassified articles; classify the right ones first; cut LLM time	Reuse the v1.1.0 relevance pipeline; add calibration and a recall-oriented threshold	Gold test set to choose the threshold
2	Category cross-check	Catch LLM category mistakes (e.g. OTHER for diversions) and flag them	The v1.1.0 category pipeline (bundle['models']['category'])	Silver + gold; >= 30 examples per category
3	Embeddings for duplicates & incident linking	Link differently-worded reports of one event; suggest duplicate incidents	Ollama nomic-embed-text (768-d, already installed) + cosine similarity	No training; tune threshold on reviewed incidents
4 (later)	Severity / time-status	Better prioritisation	Only after enough gold labels exist	Gold labels
5 (later)	Location NER	More records on the map	Gazetteer + rules first; spaCy only if justified	Annotated spans

Not in scope: training or fine-tuning LLMs, deep-learning pipelines needing a GPU, or replacing the LLM extractor. Improve the existing simple model until it beats the keyword pre-filter on the gold test set, then add tasks.

7. Target architecture: extending the rdi.services.ml package

```
backend/rdi/services/ml/
  __init__.py
  model.py           EXISTS: training_rows(), train(), load(), ensure_model(), predict_output(), status()
  seed_labels.jsonl  EXISTS: bootstrap labels
  registry.py        NEW: versioned artifacts + active version per task (replaces the single ml_classifier.joblib;
                     keep model.load()/ensure_model() working as thin wrappers so the engine/fallback code is untouch
  scoring.py         NEW: score_batch(db, ids): relevance/category scores -> ml_predictions rows (advisory)
  embeddings.py      NEW: embed(texts) via Ollama /api/embed, cache in article_embeddings table
backend/ml_training/                          (offline; not imported by the server)
  export_dataset.py   section 4.3
  evaluate.py         metrics on the gold test set for ML vs keyword pre-filter vs LLM, writes the model card
backend/tests/test_6_ml_scoring.py, test_7_ml_embeddings.py
road-disruption-intel/docs/ML_MODEL_CARD.md   generated/updated by evaluate.py (real numbers only)
```

6.1 Artifact layout and registry

```
runtime/models/relevance/2026-10-20_tfidf_lr_v1/
    model.joblib         sklearn Pipeline (vectorizer + classifier + calibration)
    metadata.json        {"task": "relevance", "version": "2026-10-20_tfidf_lr_v1",
                          "created_at": "...", "trained_on": {"silver": 2300, "gold": 240},
                          "labels": ["RELEVANT", "NOT_RELEVANT"], "threshold": 0.42,
                          "metrics_gold_test": {"precision": ..., "recall": ..., "f1": ...},
                          "code_commit": "<git sha>", "sklearn_version": "1.5.2"}
runtime/models/active.json   {"relevance": "2026-10-20_tfidf_lr_v1", "category": null}
runtime/models/ml_classifier.joblib   v1.1.0 single-file model - migrate it into this layout
```

```
# backend/rdi/services/ml/registry.py  (sketch)
import json, logging, threading
from pathlib import Path
from ...config import settings
log = logging.getLogger(__name__)
_lock, _cache = threading.Lock(), {}

def models_dir() -> Path:
    return settings.runtime_dir / "models"

def ml_available() -> bool:
    try:
        import sklearn, joblib  # noqa: F401  (in requirements.txt since v1.1.0; still import lazily)
        return True
    except ImportError:
        return False

def active_version(task: str) -> str | None:
    f = models_dir() / "active.json"
    return json.loads(f.read_text()).get(task) if f.exists() else None

def load(task: str):
    """Return (model, metadata) for the active version, or (None, None). Never raises."""
    version = active_version(task)
    if not version or not ml_available():
        return None, None
    with _lock:
        if (task, version) not in _cache:
            import joblib
            d = models_dir() / task / version
            try:
                _cache[(task, version)] = (joblib.load(d / "model.joblib"),
                                           json.loads((d / "metadata.json").read_text()))
            except Exception:
                log.exception("Could not load ML model %s/%s", task, version)
                return None, None
        return _cache[(task, version)]
```

joblib/pickle files execute code when loaded. Only load model files produced by your own training scripts from runtime/models; never accept model uploads through the web API.

8. Database changes and migration

Important: `Base.metadata.create_all()` in `db/session.py` creates **new tables** but never adds **columns** to existing tables. Prefer new tables (below). If you must add a column to an existing table, bump `SCHEMA_VERSION` and register a migration function - init_db() backs up the database before running it.

The v1.1.0 engine/fallback stores its results as ordinary Classification versions (provider 'ml') - no schema change was needed. Advisory scores for **all** articles are different: they must not create classification versions. Add two new tables to `backend/rdi/db/models.py` (created automatically on next start):

```
class MLPrediction(Base):
    """Advisory prediction of one model for one article. Never overwrites LLM/human output."""
    __tablename__ = "ml_predictions"
    __table_args__ = (UniqueConstraint("article_id", "task", "model_version", name="uq_ml_pred"),)
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    article_id: Mapped[str] = mapped_column(String(32), ForeignKey("articles.id", ondelete="CASCADE"), index=True)
    task: Mapped[str] = mapped_column(String(30), index=True)          # relevance / category
    model_version: Mapped[str] = mapped_column(String(80))
    label: Mapped[str | None] = mapped_column(String(30), index=True)
    probability: Mapped[float | None] = mapped_column(Float)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)

class ArticleEmbedding(Base):
    __tablename__ = "article_embeddings"
    article_id: Mapped[str] = mapped_column(String(32), ForeignKey("articles.id", ondelete="CASCADE"), primary_key=Tr
    model: Mapped[str] = mapped_column(String(80), primary_key=True)   # e.g. ollama/nomic-embed-text
    dim: Mapped[int] = mapped_column(Integer)
    vector: Mapped[bytes] = mapped_column(LargeBinary)                 # float32 little-endian
    created_at: Mapped[datetime] = mapped_column(DateTime, default=utcnow)
```

Optional fast filter column (only if querying ml_predictions is too slow): add `articles.ml_relevance_score` via a migration:

```
# backend/rdi/db/session.py
SCHEMA_VERSION = 2

def _m2_ml_columns(eng):
    with eng.begin() as con:
        cols = {r[1] for r in con.exec_driver_sql("PRAGMA table_info(articles)")}
        if "ml_relevance_score" not in cols:
            con.exec_driver_sql("ALTER TABLE articles ADD COLUMN ml_relevance_score FLOAT")
            con.exec_driver_sql("CREATE INDEX IF NOT EXISTS ix_articles_ml_rel ON articles (ml_relevance_score)")

MIGRATIONS: dict[int, callable] = {2: _m2_ml_columns}
# ...and add  ml_relevance_score: Mapped[float | None] = mapped_column(Float)  to Article in models.py
```

Update `DATA_DICTIONARY.md` for every new table/column.

9. Task 1 - Relevance scoring for triage (advisory)

The relevance pipeline already exists (section 5). What is missing is (a) honest evaluation, (b) calibrated probabilities with a recall-oriented threshold, and (c) scoring every article, not only the ones being classified. The sketch below shows a stricter offline training/evaluation path; fold its improvements (calibration, time- and incident-grouped split, threshold choice) into model.train() or a versioned successor.

9.1 Training and evaluation (offline)

```
# backend/ml_training/train_relevance.py  (sketch - keep it this simple for v1)
import json, datetime, subprocess
from pathlib import Path
import joblib, numpy as np
from sklearn.pipeline import Pipeline, FeatureUnion
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import precision_recall_fscore_support, average_precision_score
from rdi.config import settings

rows = [json.loads(l) for l in open(settings.runtime_dir / "ml/datasets/articles.jsonl", encoding="utf-8")]
lab = [r for r in rows if r["relevance"]]
y = lambda r: 1 if r["relevance"] == "RELEVANT" else 0
gold = [r for r in lab if r["label_source"].startswith("gold")]
# time-based, incident-grouped split: newest 30% of GOLD = test; never put one incident on both sides
gold.sort(key=lambda r: r["published_at"] or "")
cut = int(len(gold) * 0.7)
test = gold[cut:]
test_groups = {r["incident_id"] for r in test if r["incident_id"]}
train = [r for r in lab if r not in test and (r["incident_id"] is None or r["incident_id"] not in test_groups)]

features = FeatureUnion([
    ("word", TfidfVectorizer(ngram_range=(1, 2), min_df=2, sublinear_tf=True)),
    ("char", TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5), min_df=3, sublinear_tf=True)),
])
clf = CalibratedClassifierCV(LogisticRegression(max_iter=2000, class_weight="balanced"), cv=3)
pipe = Pipeline([("features", features), ("clf", clf)])
pipe.fit([r["text"] for r in train], [y(r) for r in train])

p = pipe.predict_proba([r["text"] for r in test])[:, 1]
# choose threshold for high RECALL (triage must not drop real disruptions), e.g. recall >= 0.95 on gold test
...  # compute precision/recall/F1/average precision, compare with the keyword pre-filter baseline
version = datetime.date.today().isoformat() + "_tfidf_lr_v1"
out = settings.runtime_dir / "models/relevance" / version
out.mkdir(parents=True, exist_ok=True)
joblib.dump(pipe, out / "model.joblib")
(out / "metadata.json").write_text(json.dumps({...}, indent=2))   # include REAL metrics + counts + git sha
```

If there is no gold test set yet, the script must say so in metadata ("metrics_gold_test": null, "evaluation": "silver only - not a measure of accuracy") and the UI must show that warning.

9.2 Inference (online)

```
# backend/rdi/services/ml/scoring.py  - reuse the existing model bundle
from . import model as ml
def predict_relevance(texts: list[str]) -> tuple[str | None, list[float]]:
    bundle = ml.load()                        # None if no model / incompatible file
    if bundle is None:
        return None, []
    pipe = bundle["models"]["relevance"]
    i = list(pipe.classes_).index("RELEVANT")
    return bundle["meta"]["version"], [float(p) for p in pipe.predict_proba(texts)[:, i]]
```

9.3 Where it is wired in

Hook	File / function	Change
New articles	services/ingest.py: Ingestor.ingest()	Do NOT call the model per row inside ingest (keeps imports fast and transactions short). Instead collect new article ids and let the caller enqueue an 'ml_score' job (acquisition.run_source already collects new_ids; importer can do the same at batch end).
Backfill	services/job_handlers.py: new @handler("ml_score")	Score articles in batches of 500 (one predict_proba call per batch), write MLPrediction rows (+ ml_relevance_score), commit per batch, honour ctx.cancelled().
Queue priority	job_handlers.select_articles_for_classification()	Allow sort by ml_relevance_score desc and filter ml_relevance_min.
Auto-classify	acquisition._maybe_auto_classify()	Optional setting: classify new articles whose ML score >= threshold (in addition to / instead of the keyword pre-filter).
Records API	api/records.py: apply_record_filters(), SORTS, record_row()	Add filter ml_relevance_min, sort key ml_relevance, field ml_relevance in rows and exports.

```
# backend/rdi/services/job_handlers.py  (sketch)
@handler("ml_score")
def _ml_score(ctx: JobContext, params: dict) -> dict:
    from .ml.scoring import score_batch
    with SessionLocal() as db:
        ids = params.get("article_ids") or list(db.scalars(select(Article.id)))
    done = 0
    for i in range(0, len(ids), 500):
        if ctx.cancelled():
            break
        with SessionLocal() as db:
            score_batch(db, ids[i:i + 500])      # writes MLPrediction rows, no LLM calls
            db.commit()
        done += len(ids[i:i + 500])
        ctx.update(processed=done, total=len(ids), message=f"{done}/{len(ids)} scored")
    return {"message": f"Scored {done} article(s)"}
```

10. Task 2 - Category cross-check

The category pipeline already exists in the v1.1.0 bundle (`bundle["models"]["category"]`). Improve it with gold labels (merge rare categories into OTHER if they have fewer than ~30 examples) and use it in two places:

- **Disagreement flag:** in `classifier.review_reasons()` add a reason when the ML model's top category differs from the LLM's category and the ML probability is high (e.g. >= 0.8): *"ML model suggests DIVERSION (0.86) - LLM said OTHER"*. Gate it behind a setting.
- **Suggestion in the record panel:** show 'ML suggestion: DIVERSION (86%)' in `RecordDrawer.tsx`; the operator applies it with Edit fields. Never change the category automatically.

```
# backend/rdi/services/classification/classifier.py  inside review_reasons(), after existing checks
ml = settings_store.get(db, "ml") if db is not None else {}   # pass db in, or read ccfg["ml_cross_check"]
if cls.relevance == "RELEVANT" and ml.get("category_cross_check"):
    from ..ml.category import predict_category
    version, preds = predict_category([article_source_text(article)])
    if preds:
        label, prob = preds[0]
        if label != cls.category and prob >= ml.get("cross_check_min_prob", 0.8):
            reasons.append(f"ML model suggests {label} ({prob:.0%}) - LLM said {cls.category}")
```

review_reasons() currently has the signature (cls, article, ccfg, issues). The cleanest change is to read the ML settings into ccfg in classify_article() before calling it, so the function signature stays pure.

11. Task 3 - Embeddings for duplicates and incident linking

Ollama is already installed on the client PC with **nomic-embed-text** (verified: returns 768-dimension vectors via POST http://127.0.0.1:11434/api/embed). Using it avoids adding PyTorch.

```
# backend/rdi/services/ml/embeddings.py  (sketch)
import struct, httpx
from ..settings_store import get as get_setting

def embed(db, texts: list[str], client: httpx.Client | None = None) -> list[list[float]]:
    cfg = get_setting(db, "ml")
    base = get_setting(db, "llm")["providers"]["ollama"]["base_url"]
    c = client or httpx.Client(timeout=60)
    r = c.post(f"{base}/api/embed", json={"model": cfg.get("embedding_model", "nomic-embed-text"), "input": texts})
    r.raise_for_status()
    return r.json()["embeddings"]

def pack(v: list[float]) -> bytes:   return struct.pack(f"<{len(v)}f", *v)
def unpack(b: bytes) -> list[float]: return list(struct.unpack(f"<{len(b)//4}f", b))
```

Where to use it

- **Incident linking** - `services/incidents.py: find_match()`. Today a candidate matches only if category + city + time window agree AND specific roads/localities agree. Add an optional similarity check: compute cosine similarity between the new article and the incident's linked articles; require similarity >= a tuned threshold (e.g. 0.80) **in addition to** category/city/time. Embeddings must only make linking stricter or resolve 'not comparable' cases - never merge different cities or categories.
- **Duplicate incident suggestions** - `api/dashboard.py: data_quality()` currently suggests pairs by category + city + 48 h. Add the similarity score and sort by it.
- **Near-duplicate articles** - optional: same event, different publisher is NOT a duplicate (keep both as evidence); only same-source near-identical texts may be marked near_duplicate_of.

Tune thresholds on incidents operators have reviewed (merged / detached actions are recorded in audit_log). Embedding calls are slow-ish on CPU: compute them in the 'ml_score' job, cache in article_embeddings, and never block an API request on them. If Ollama is unreachable, skip silently and keep current behaviour.

12. API, settings, background jobs and user interface

Already present: settings 'classification.engine', 'ml_fallback', 'ml_relevance_threshold'; endpoints GET /api/ml/status, POST /api/ml/train (job 'ml_train'), POST /api/ml/test; the Classification engine page. Add the following for the remaining tasks.

12.1 Settings - services/settings_store.py

```
DEFAULTS["ml"] = {
    "enabled": False,                   # off until a model is trained and evaluated
    "relevance_auto_classify_min": None, # e.g. 0.6 to auto-classify new articles above this score
    "category_cross_check": False,
    "cross_check_min_prob": 0.8,
    "embedding_model": "nomic-embed-text",
    "use_embeddings_for_linking": False,
    "link_similarity_min": 0.80,
}
# and add "ml" to EDITABLE_SETTING_KEYS in api/config_api.py
```

12.2 API - extend the /api/ml/* endpoints in api/config_api.py (or move them to a new api/ml_api.py)

Method / path	Purpose
GET /api/ml/status (exists)	Extend: active model per task, gold metrics or the 'no gold evaluation' warning, last scoring job

Method / path	Purpose
POST /api/ml/score	Start 'ml_score' job: {article_ids?} or all unscored articles; returns job_id
POST /api/ml/embed	Start embedding job (optional)
POST /api/ml/activate	{task, version} - switch active model (writes runtime/models/active.json, audit_log entry)
GET /api/records?ml_relevance_min=0.7&sort=ml_relevance	Filter/sort records by ML score (extend api/records.py)

v1.1.0 allows retraining the simple model from the UI because it only learns from the application's own labels. Keep activation of a new model version an explicit operator action, and never accept uploaded model files.

12.3 Frontend

- `src/app/pages/LLMSettings.tsx` - extend the existing EnginePanel: gold metrics, model versions/activation, 'Score all articles' (uses the JobProgress component), toggles for the new ml settings.
- `src/app/pages/Records.tsx`: add an 'ML relevance' column (percentage badge) and a 'Min ML score' filter; include the param in the URL like the other filters (useFilterParams LIST_KEYS/SCALAR_KEYS).
- `src/app/components/RecordDrawer.tsx`: show 'ML relevance 0.91 (model v...)' and 'ML category suggestion' under Classification, clearly labelled as advisory.
- `src/app/pages/Overview.tsx`: optional tile 'Unclassified with high ML score' linking to the filtered records list.
- Rebuild the UI after changes: `cd frontend && npm ci && npm run build` and commit `frontend/dist` (the client has no Node.js).

13. Dependencies, Windows setup and model artifacts

```
# backend/requirements.txt  (since v1.1.0 - installed by setup_and_start.bat)
scikit-learn>=1.5,<2.0        # brings numpy, scipy and joblib
```

- Existing installations get it by running `setup_and_start.bat` once. Verified on the client PC: Python 3.12, scikit-learn 1.9.1. Check that wheels exist for Python 3.11-3.14 before adding packages.
- Keep imports of sklearn/joblib lazy inside functions (as model.py does) so the API starts fast and survives a broken installation; status() reports 'sklearn_installed'.
- A model saved by another scikit-learn version may fail to load: model.load() then returns None and ensure_model() retrains automatically. Keep that behaviour for versioned artifacts.
- `runtime/` is already gitignored, so trained models are not committed. To ship a model to the client, attach it to a GitHub Release (zip of runtime/models/<task>/<version>) - not to the repository.
- Record the scikit-learn version in metadata.json; refuse to load a model trained with a different major version.

14. Evaluation protocol and model card

- **Gold test set only** for reported metrics (operator-reviewed). Silver (LLM) agreement may be reported separately and must be labelled 'agreement with llama3:8b, not accuracy'.
- **Split by time** (train on older, test on newer) and keep all articles of one incident on the same side.
- **Baselines:** compare against (a) the keyword pre-filter (LIKELY_RELEVANT = positive) and (b) the LLM itself on the same gold test.
- **Relevance metrics:** precision, recall, F1, average precision; choose the operating threshold for recall >= 0.95 (triage must not hide real disruptions) and report the precision at that threshold.
- **Category metrics:** per-class precision/recall/F1, macro-F1, confusion matrix; report class counts.
- **Linking:** pairwise precision/recall of 'same incident' decisions on reviewed incidents; report how many merges the embedding rule would have caught and how many wrong links it would add.
- **Speed:** articles scored per second on the client CPU.

- Write everything to `docs/ML_MODEL_CARD.md`: data counts, label sources, splits, metrics with confidence intervals (bootstrap), known failure modes, date, git commit. No hand-edited numbers.

15. Tests and definition of done

15.1 Tests - backend/tests/test_6_ml_*.py

The test suite uses a throw-away database (tests/conftest.py) and mocks all HTTP. ML tests must not need a GPU, Ollama or downloaded models. The existing fixture 'trained' in test_5_highways_ml.py trains the v1.1.0 model on the seed set in a few seconds - reuse it.

```
import pytest, httpx

def test_registry_returns_none_without_model(): ...          # app works with no trained model
def test_train_tiny_relevance_model_and_score(db, tmp_path): ...  # train on ~20 fixture texts, save, load, score
def test_ml_score_job_writes_predictions_and_is_idempotent(db): ...
def test_ml_never_changes_llm_or_human_output(db): ...        # classification rows untouched
def test_category_disagreement_flag(db): ...                 # review_reasons adds the ML reason only when enabled
def test_embeddings_via_mocked_ollama(db): ...               # httpx.MockTransport for /api/embed
def test_linking_with_embeddings_never_merges_different_cities(db): ...
def test_ml_status_endpoint_reports_no_gold_evaluation(client): ...
def test_records_filter_ml_relevance_min(client): ...
```

Run: `cd backend && ..\.venv\Scripts\python -m pytest tests -q` (expected: 75 existing + new tests, 0 failed).

15.2 Definition of done

#	Criterion
1	All existing tests pass; new ML tests pass; frontend type check and build pass.
2	App starts and works with no model trained and with an unloadable model file; engine/fallback behaviour of v1.1.0 unchanged.
3	A gold evaluation set of >= 300 operator-reviewed articles exists (or the gap is clearly documented).
4	docs/ML_MODEL_CARD.md generated from real runs, comparing ML vs keyword pre-filter vs LLM on the gold test set.
5	ML predictions stored with model version; LLM and human classifications never modified by ML.
6	Records list/export show ML score; filter and sort work server-side; UI labels ML as advisory.
7	'ml_score' job is batched, cancellable, resumable, and does not hold the DB write lock for long.
8	Embedding-based linking is behind a setting, tuned on reviewed incidents, and covered by tests.
9	README, USER_GUIDE (new section + screenshot), DATA_DICTIONARY, API.md, CHANGELOG and TEST_REPORT updated.
10	No secrets, runtime DB or model binaries committed; work done on a feature branch with a pull request.

16. Ready-to-paste prompt for Claude / ChatGPT

Upload this PDF to the project, give the assistant the repository (clone, zip or the files named in section 2), then paste:

```
You are working on the repository github.com/AnandRawat138/road-disruption-intelligence
(Road Disruption Intelligence, FastAPI + SQLAlchemy/SQLite backend in road-disruption-intel/backend/rdi,
React/TypeScript frontend in road-disruption-intel/frontend). The attached "ML Integration Guide" PDF is the
specification. Follow it exactly.

A baseline ML layer already exists (v1.1.0): rdi/services/ml/model.py (TF-IDF + logistic regression for
relevance, category, time status), the classification engine setting (llm|ml), the automatic ML fallback
and tests/test_5_highways_ml.py. Extend it - do not rebuild it - in this order, one pull request each:
  M1  ml_training/export_dataset.py + evaluate.py: gold set export, metrics for ML vs keyword pre-filter
      vs LLM on the gold test set, docs/ML_MODEL_CARD.md. Calibrate the relevance model and choose a
      recall-oriented threshold.
  M2  services/ml/registry.py (versioned artifacts, keep model.load()/ensure_model() as wrappers),
      services/ml/scoring.py, MLPrediction table, "ml_score" job, /api/ml/score, tests.
  M3  Records API + UI: ML score column, filter, sort, export field; EnginePanel extensions; record panel.
  M4  Category cross-check review flag (behind the "ml" settings), tests.
  M5  Embeddings via Ollama nomic-embed-text, article_embeddings table, optional similarity check in
      incidents.find_match() and duplicate suggestions, tests; documentation updates (README, USER_GUIDE,
      DATA_DICTIONARY, API.md, CHANGELOG, TEST_REPORT).

Rules:
- Read the existing code before changing it; match its style (type hints, small functions, docstrings
  where the surrounding code has them). Do not rewrite unrelated code.
- ML never auto-approves and never replaces an LLM or human classification. New ML scores are advisory
  (own table). Keep the v1.1.0 rule: the engine/fallback writes ML classifications only for records
  without an LLM/human result, unless the operator explicitly reclassifies with ML.
- The app must run without a trained model and with an unloadable model file.
- No PyTorch/TensorFlow; CPU-only; Windows; Python 3.11-3.14.
- Do not invent labels or metrics. There is currently 1 approved record and no corrections: report
  that honestly and use silver (LLM) labels only for training, never as proof of accuracy.
- Keep SQLite transactions short (commit per batch); never hold a write transaction during slow work.
- All existing tests must pass; add tests that run without network (use httpx.MockTransport).
- Explain each change: file, function, why. At the end list what was run (tests, build) with real output.
Start with M1. First summarise the relevant existing code (services/ml/model.py, classifier.py,
job_handlers.py, ingest.py, db/models.py, db/session.py) to confirm your understanding, then propose the
exact diff.
```

17. Pitfalls to avoid

Pitfall	Why it is a problem / what to do instead
Using data/weak_labels as labels	It has no labels - only a constant 0.85 confidence.
Using the 'city' field from data/*.jsonl as location	It is the city that was searched for, not where the event happened (e.g. 'National Highway' appears as a city).
Evaluating on LLM labels and calling it accuracy	It only measures agreement with llama3:8b. Use the gold set.
Re-implementing the existing classifier	model.py, the engine switch, fallback, API and UI exist. Extend them; keep tests in test_5_highways_ml.py green.
Random train/test split	The same story appears from several publishers and queries; random splits leak. Split by time and by incident.
Dropping articles the keyword pre-filter calls UNLIKELY	Then recall can never be measured or improved. Sample them into the gold set.
Calling the model inside Ingestor.ingest() per row	Slows imports of 78k rows and lengthens transactions. Batch in a job.
Merging reports from different publishers as duplicates	They are independent evidence for one incident; link them, do not deduplicate them.
Overwriting the LLM category with the ML one	Breaks provenance and audit. Store separately, flag for review.
Shipping .joblib files in Git / loading untrusted pickles	Large binaries bloat the repo; pickles execute code. Use GitHub Releases and only your own artifacts.
Treating model probability as certainty	Calibrate, document the threshold, show it as advisory.
Long-running work in API requests	Use the job runner (services/jobs.py) like import/classify/ml_train do.
Training on the ML model's own outputs	Feedback loop that amplifies its mistakes. training_rows() excludes provider 'ml' - keep it that way.

Pitfall	Why it is a problem / what to do instead
Quoting the 96% relevance figure as accuracy	It is cross-validated agreement with labels that include easy weak negatives; on LLM-labelled rows macro-F1 is 0.63.
Letting ML results silently replace LLM results	Breaks provenance. The fallback fills gaps only; replacement requires an explicit operator reclassify.

Questions about the existing behaviour can be answered from the repository documentation: README.md, road-disruption-intel/DATA_DICTIONARY.md, docs/API.md, AUDIT_REPORT.md and TEST_REPORT.md.